

UNIT 3 – Models, Database, ORM, Migrations & CRUD

Exhaustive Study Notes covering Database Design, Field Types, Relationships, QuerySets & Migrations

1. Django Model

A Django Model is a Python class representing a relational database table. Attributes of the class correspond to table columns, and instances of the class correspond to database table rows.

2. ORM

The Object-Relational Mapper (ORM) abstracts raw SQL operations into Python methods. ORM benefits include database engine independence (switch from SQLite to PostgreSQL without code edits) and automatic SQL injection sanitization.

Django ORM Abstraction & SQL Translation Engine



Diagram: Django ORM Abstraction and SQL Query Compilation Architecture.

3. Creating a Model

Models inherit from `django.db.models.Model`:

```
from django.db import models

class Department(models.Model):
    name = models.CharField(max_length=100)
    code = models.CharField(max_length=10, unique=True)

    def __str__(self):
        return self.name
```

4. CharField

Stores short to medium string text. Requires mandatory `max_length` parameter.

5. TextField

Stores large multiline text strings without length restrictions (rendered as `<textarea>` in forms).

6. IntegerField

Stores standard 32-bit signed integers.

7. EmailField

Stores email strings and uses `EmailValidator` to verify valid address format.

8. BooleanField

Stores true/false values as SQL `BOOLEAN` or `TINYINT` columns.

9. DateField and DateTimeField

Stores date and timestamp objects. Options: • `auto\_now\_add=True`: Automatically sets timestamp when record is created. • `auto\_now=True`: Automatically updates timestamp whenever record is saved.

10. FileField and ImageField

Stores file path references in DB while saving raw binary files to media directory. `ImageField` requires Python `Pillow` library installation.

```
doc = models.FileField(upload_to='documents/')
photo = models.ImageField(upload_to='profile_photos/')
```

11. Primary Key

By default, Django automatically creates an auto-incrementing `id = models.BigAutoField(primary\_key=True)` unless an explicit primary key is declared.

12. null vs blank

• `null=True`: Database-level option. Allows SQL database column to store `NULL` values. • `blank=True`: Validation/Form-level option. Allows field to be left empty in forms and admin panel.

13. default

Specifies a fallback default value assigned when no explicit field value is provided.

14. unique

Enforces database unique column constraint (`unique=True`), preventing duplicate field entries.

15. One-to-One

Creates strict 1:1 model link using `OneToOneField`. Common for user extensions (Profile model).

16. ForeignKey

Creates Many-to-One relationship where multiple child records link to a single parent record.

17. ManyToManyField

Creates Many-to-Many relationship. Django automatically generates an underlying join/junction table.

Django Database Model Relationships (ER Model)

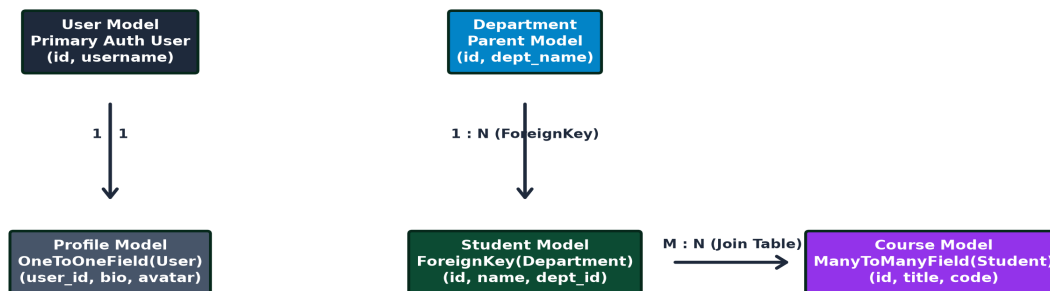


Diagram: Database Model Relationships ER Diagram (OneToOne, ForeignKey, ManyToMany).

18. on_delete

Defines referential integrity actions when a referenced parent record is deleted: • `CASCADE` : Deletes all dependent child records automatically. • `PROTECT` : Prevents deletion of parent if child records exist (raises `ProtectedError`). • `SET_NULL` : Sets foreign key column to `NULL` (requires `null=True`). • `SET_DEFAULT` : Resets foreign key to default value.

19. Migrations

Migrations track changes made to `models.py` and synchronize schema state with the SQL database.

20. makemigrations

Scans `models.py` files and writes new Python migration files inside `migrations/` folder.

21. migrate

Executes unapplied migration files and runs `CREATE TABLE` / `ALTER TABLE` DDL queries against the database.

Django Migrations Schema Versioning Workflow



Diagram: Django Migrations Schema Versioning & Execution Flowchart.

22. Admin Model Registration

Registering models with `ModelAdmin` customization class:

```
from django.contrib import admin
from .models import Student

@admin.register(Student)
class StudentAdmin(admin.ModelAdmin):
    list_display = ('name', 'email', 'department', 'age')
    list_filter = ('department',)
    search_fields = ('name', 'email')
```

23. Create

ORM Creation: `Student.objects.create(name='Asha', age=20)` or `s = Student(...); s.save()`.

24. Read

ORM Retrieval: `Student.objects.all()`, `.get(id=1)`, `.filter(age\_\_gte=18)`.

25. Update

ORM Updating: `student = Student.objects.get(id=1); student.age=21; student.save()` or `QuerySet.update(age=21)`.

26. Delete

ORM Deletion: `student.delete()` or `Student.objects.filter(age\_\_lt=18).delete()`.

27. QuerySet

A QuerySet represents a collection of database objects. QuerySets are lazily evaluated—no SQL query is executed until data is evaluated.

28. filter()

Returns a new QuerySet containing objects matching kwargs criteria.

29. get()

Returns a single matching model instance. Raises `DoesNotExist` if 0 matches or `MultipleObjectsReturned` if >1 match.

30. all()

Returns a QuerySet of all records in the table.

31. exclude()

Returns a QuerySet excluding objects matching kwargs criteria.

32. order_by()

Sorts QuerySet: `order\_by('name')` (ascending) or `order\_by('-created\_at')` (descending).

33. Lookup Operators

Field lookups using double underscores (`\_\_`):

- `\_\_exact` / `\_\_iexact`: Case sensitive / insensitive equality.
- `\_\_contains` / `\_\_icontains`: Substring search.
- `\_\_gt`, `\_\_gte`, `\_\_lt`, `\_\_lte`: Numerical comparison (>, >=, <, <=).
- `\_\_in`: Check if value exists in a provided list.
- `\_\_range`: Check if value lies between start and end bounds.

```
adults = Student.objects.filter(age__gte=18)
search = Student.objects.filter(name__icontains='asha')
```

34. Database Support

Django supports SQLite (default), PostgreSQL (`psycopg2`), MySQL (`mysqlclient`), MariaDB, and Oracle by changing `ENGINE` in `settings.DATABASES`.